BROUGHT TO YOU BY RUSTACEANSKENYA

Transitioning to Rust

The Learning Curve

Presented by

Michael Kinuthia

SOFTWARE ENGINEER & SYSTEMS ARCHITECT

April 24, 2026

Session Overview

Why Rust Feels Hard

Unlearning old habits

The Ownership Model

Making the memory model click

Practical Strategies

How to stop fighting the compiler

Tools & Habits

rust-analyzer, clippy, and cargo

Your Learning Roadmap

The 3-month plan to proficiency

Q&A and Community

Live troubleshooting

⚠️

Why Rust Feels Hard

Spoiler: It's not you, it's a completely different paradigm.

The Real Challenge Isn't Syntax

Most languages hide memory management.
Rust forces you to architect it upfront.

Ownership

Every single value has exactly ONE owner at a time. No exceptions.

Lifetimes

Data doesn't live forever. You cannot reference a value that has been destroyed.

Explicit Control

No hidden garbage collection pauses. No silent, expensive deep copies.

How Rust Compares

Language	Memory Management	When Bugs Hit
Python/JS	Garbage Collected	**Production** 💥
C/C++	Manual (malloc/free)	**Runtime** 🔥
Rust	**Ownership Model**	**Compile Time** ✅

The borrow checker isn't punishing you; it's teaching you what other languages silently get wrong.

Common Frustrations

"Fighting the borrow checker instead of building features"

"Feeling unproductive compared to my main language"

"Error messages that look like alien hieroglyphics"

"My logic is perfect, but the compiler says NO"

THIS IS TEMPORARY. EVERYONE GOES THROUGH THIS.
THE "success!" MOMENT IS COMING.

The Ownership Model

The Library Book Analogy

The Move (Unlearning Old Habits)

The Rule

You check out a book. You are the sole owner. If you give the book to a friend, you can no longer read it.

This prevents double-free errors where two variables try to delete the same memory.

The Code

```
let original = String::from("Data");

// The "Move" happens here
let new_owner = original;

// COMPILER PANIC! original is gone.
println!("{}", original);
```

The Two Core Borrowing Rules

RULE 1: IMMUTABLE

Infinite Readers Allowed

```
let r1 = &data;
let r2 = &data;
println!("{}, {}", r1, r2);
```

✓ Friends can read over your shoulder.

RULE 2: MUTABLE

Exactly ONE Writer Allowed

```
let w1 = &mut data;
// let w2 = &mut data; // ERROR!
*w1 += 1;
```

✓ Only one person can hold the pen.

🔧

Practical Strategies

Escaping Tutorial Hell

Strategy 1: Start Small

❌ Don't Try To...

- Rewrite a massive production API
- Build a complex Game Engine
- Start immediately with Async/Await

✅ Instead, Build...

- **A CLI Calculator or TODO app**
- **A fast CSV/JSON File Parser**
- **A Web Scraper using `reqwest`**

Strategy 2: Read The Compiler Errors

The compiler is not blocking you. It is your Pair Programmer.

```
error[E0502]: cannot borrow `data` as mutable because it is also borrowed as immutable
  --> src/main.rs:6:5
   |
 4 |     let reader = &data;
   |                  ----- immutable borrow occurs here
 5 |
 6 |     let writer = &mut data;
   |     ^^^^^^^^^^ mutable borrow occurs here
 7 |
 8 |     println!("{}", reader);
   |                    ------ immutable borrow later used here
```

Solution: The compiler literally draws arrows to your exact logical flaw. Read the `help:` tags!

Strategy 3: Clone Is Your Friend

Make it work. → Make it idiomatic. → Make it fast.

When learning, do not spend 3 hours fighting lifetime annotations `&'a str`.

Just use **.clone()** and move on.

You can optimize for performance later once the core logic compiles.

The Right Resources

📘 Reading

- The Rust Programming Language
- Rust for Rustaceans
- Programming Rust

⚡ Doing

- Rustlings (Interactive)
- Exercism Rust Track
- Rust by Example

👥 Asking

- Rust Kenya Community!
- r/rust on Reddit
- users.rust-lang.org

Tools & Habits

Setting up for Success

Your Development Stack

rust-analyzer

The ultimate LSP for VS Code/Neovim.

Shows types inline, explains lifetimes, and suggests automatic fixes.

cargo clippy

The strict, helpful linter.

It teaches you how to write "idiomatic" Rust patterns.

cargo-watch

Instant feedback loop.

```
cargo watch -x run
```

cargo-edit

Easy dependency management.

```
cargo add serde
```

Habits for Success

- Always start with `cargo new`, don't write bare files.
- Use the `dbg!()` macro to inspect variables quickly.
- Write tests early! Cargo makes unit testing incredibly easy.
- Pair program. Explaining your error aloud usually solves it.

30 minutes daily beats 5 hours once a week.

Your Learning Roadmap

From Beginner to Confident Engineer

The 3-Month Plan

Month 1

Foundations

- Complete `rustlings`
- Read Chapters 1–10
- Build 2 CLI tools
- **Master Ownership**

Month 2

Practical Projects

- Build a REST API
- **Master `Result` & `Option`**
- Choose an ecosystem

Month 3

Architecture

- **Learn Lifetimes properly**
- Smart Pointers
- Read open-source code

Milestones to Celebrate 🎉

- ☑ The first time your code compiles on the first try.
- ☑ Fixing a borrow checker error without Googling it.
- ☑ When you realize Rust just prevented a massive production bug.
- ☑ Your first merged Pull Request to a Rust project.

Key Takeaways

- ★ Rust isn't 'harder' — it's just explicit about what other languages hide.
- ★ The compiler is your senior engineer. Listen to it.
- ★ Start small. CLI tools before full-stack frameworks.
- ★ Use `.clone()` freely while you learn the ropes.

Your Next Steps

1. Connect

Join the Rust Kenya Discord and find a learning buddy from tonight's session.

2. Commit

Pick one starter project tonight. Just run `cargo new` to break the ice.

We are here to support you!

Questions & Discussion

Drop your biggest Rust blockers in the chat.

Thank You!

Let's keep building.

CONNECT WITH MICHAEL

𝕏 @Mking708

💻 github.com/michaelmk708

🔗 in/michaelkinuthia708

RUSTKENYA

🌐 https://rust.ke/